SystemVerilog · Class 기반 검증 환경

HW sv adder
코드 분석 및 시뮬레이션

transaction → generator → driver → DUT → monitor → scoreboard, 그리고 이걸 묶는 environment



참조 코드 `system_verilog/lib/adder.sv` · `lib/sim_file/tb_adder.sv` · `tb_adder_sequential.sv`
`tb_adder_fork_join_none.sv` · `tb_adder_fork_join.sv`

시뮬레이터 Vivado 2020.2 xsim · 2026.11.04 (수) 수업 복습 자료

이 자료를 읽는 순서

파트별로 끊어 읽어도 되도록, 각 파트는 '개념 → 코드 → 결과' 순서

PART 1
기초 개념

- 왜 class(OOP)로 검증하나
- handle · new · heap
- 메모리 4구역과 tb_adder
- 전체 구조도

PART 2
컴포넌트별 코드 해석

- DUT · interface
- transaction · 제약조건
- generator · driver
- monitor · scoreboard
- environment · tb_adder top

PART 3
동기화와 실행 흐름

- fork / join 3종 비교
- TB 4개 버전 비교
- event 핸드셰이크 타임라인

PART 4
시뮬레이션 결과

- 버전별 xsim 로그
- 256회 결과 · 분포
- 주의할 점 (버그 포인트)

PART 5
복습

- 면접 단골 질문 Q&A
- 핵심 키워드 한 장 요약

왜 class(객체지향)로 검증하나

역할을 나눠 재사용성을 높이고, 객체를 필요할 때 만들고 버리면서(동적 할당) 동시에 돌린다

Verilog TB — 정적 할당

- 프로그램이 실행되는 순간 변수 위치가 고정
- 시뮬레이션이 끝날 때까지 메모리를 계속 차지
- 입력 패턴을 바꾸려면 `initial` 블록을 다시 작성
- for 문으로 값만 바꾸는 방식 → 패턴 다양화 어려움

SystemVerilog class — 동적 할당

- `new`를 실행할 때 비로소 heap에 객체가 생김
- 참조하는 handle이 없어지면 자동 회수(GC)
- 한 번 셋업해 두면 `randomize()`로 다양한 패턴 입력
- gen · drv · mon · scb가 **동시에** 동작 (fork)

객체를 쓰려면 handle이 먼저 필요하다 — handle은 '객체의 주소를 담는 변수'

transaction tr;
handle 선언 · 값 = null

→

tr = new;
heap에 객체 생성

→

가상 주소 할당
OS가 시뮬레이터 프로세스에 부여

→

page table
OS가 가상→물리 매핑 관리

→

MMU
읽기/쓰기 때 물리 주소로 변환

※ 시뮬레이터가 보는 주소(예: 0x5000)는 **가상 주소**다. 실제 물리 위치는 OS의 page table + MMU가 결정하므로 실행할 때마다 달라질 수 있다.

코드 한 줄은 메모리 어디에 놓이나

tb_adder를 구성하는 요소를 Text · Data/bss · Stack · Heap 네 구역에 매핑

구역	언제 생기나	언제 사라지나	tb_adder에서 해당하는 것
Text (코드)	프로그램 로드 시	프로그램 종료 시	class의 function · task 코드 (class 별 1벌만 존재)
Data / bss (정적)	elaboration (시뮬 시작 전)	시뮬 종료	adder_if · dut 인스턴스, module 변수 env handle
Stack	function · task 호출 시	반환될 때	new()의 인자, run()의 지역 변수 · 실행 프레임
Heap (동적)	new 실행 시	참조 handle이 0개 일 때 (GC)	env · gen · drv · mon · scb · mailbox · tr 객체

new가 실행될 때 일어나는 일

1. 시뮬레이터가 자기 프로세스의 heap에 객체 공간을 잡는다
2. 멤버를 기본값으로 초기화한다 (bit → 0, logic → x), 그 다음 사용자 `function new` 본문 실행
3. 객체의 참조값(주소)을 handle 변수에 저장한다

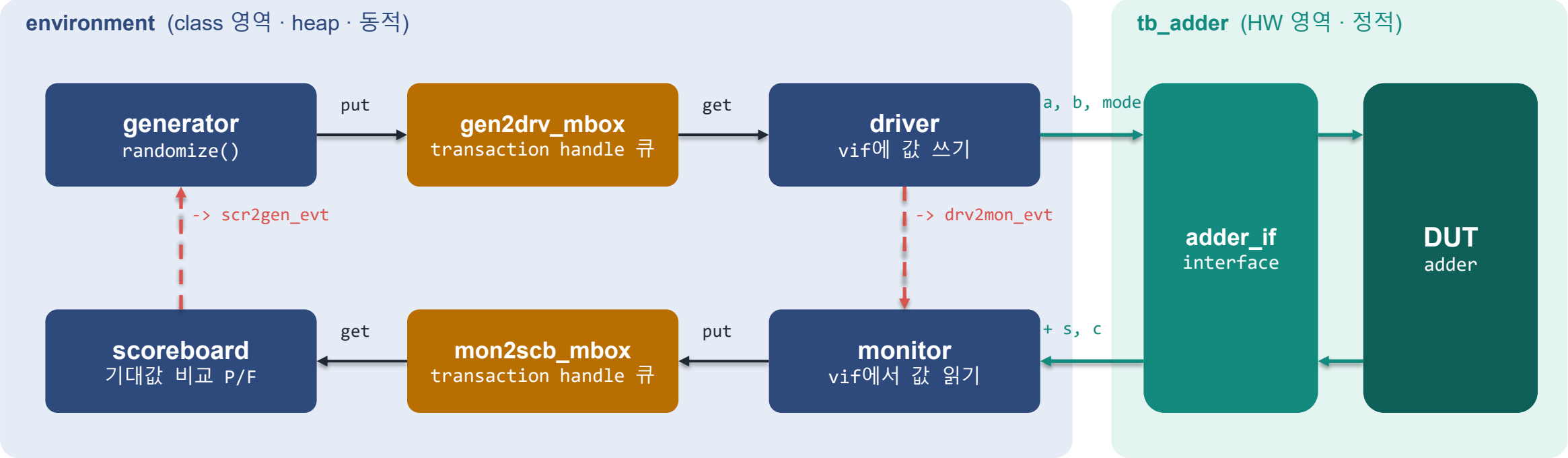
예시 — `tr = new;` 직후 heap (주소는 설명용)

0x5000	0x5008	0x5009	0x500A	0x500B	0x500C
헤더 : 타입 정보	a = 0	b = 0	mode = 0	s = x	c = x

bit (2상태) → 0으로 초기화 : drive용 `rand` 변수 **logic (4상태)** → x로 초기화 : monitor가 DUT 출력을 담는 변수

전체 검증 환경 구조도

데이터는 mailbox로, 타이밍은 event로, 하드웨어 연결은 virtual interface로 전달한다



mailbox = 무엇을

transaction 객체의 **주소(handle)**를 큐에 넣고 뺀다. 비어 있으면 `get`이 대기

event = 언제

`->evt`로 신호를 보내고 `@(evt)`로 기다린다. 순서를 강제하는 장치

virtual interface = 어디로

class(동적)가 정적 HW인 `adder_if`를 **참조**해서 핀을 읽고 쓴다

DUT(adder)와 interface

adder.sv : mode=0이면 덧셈, mode=1이면 뺄셈 · interface : DUT 핀을 한 묶음으로

adder.sv / tb_adder_*.sv

```
module adder (
  input logic [7:0] a,  // input -> 베릴로그에서는 wire 자료형
                        // input logic -> 4상태 모두 가능
  input logic [7:0] b,
  input logic mode,
  output logic [7:0] s,
  output logic c
);

  assign {c,s} = (mode)? a-b : a+b;

endmodule


interface adder_interface();
  logic [7:0] a;
  logic [7:0] b;
  logic       mode;
  logic [7:0] s;
  logic       c;
endinterface
```

1 **logic = 4상태 통합 자료형**

0 · 1 · x · z 4가지 값을 가진다

Verilog의 `wire`/`reg` 구분 없이 `input logic`, `output logic`으로 통일해서 쓴다

2 **{c,s} 9비트 결합 대입**

좌변이 9비트라 `a+b`도 9비트로 계산 → 넘친 자리(carry)가 `c`로

뺄셈은 a < b면 빌림이 생겨 `c = 1`

예) 250 + 46 = 296 → **c=1, s=40** (로그 실제 값)

3 **interface = 신호 묶음 (정적 HW)**

TB top에서 `adder_if()`로 모듈처럼 인스턴스 (괄호 필수)

DUT 포트는 `.a(adder_if.a)`처럼 `.`(접근 연산자)로 연결

class는 이 실체를 **virtual**로 참조만 한다 → driver 편

transaction — 주고받을 데이터의 자료형

스티뮬러스(입력)와 DUT 결과(출력)를 한 객체에 담는 '데이터 묶음' 설계도

tb_adder_fork_join.sv

```
class transaction;
  rand bit [7:0] a;
  rand bit [7:0] b;
  rand bit       mode;

  logic [7:0]  s;
  logic        c;

// 제약 조건
  constraint adder_limit{
    a > 128;
    b < 64;
  }
  constraint adder_inside{
    a inside {128,200,212,250};  // {[128:200]} -> 범위 지정 가능
  }
  constraint adder_distribute{
    mode dist{
      0:/80,
      1:/20
      };
    }

  function debug_print(string name);
    $display("%t :[%s] a =%d, b = %d, mode = %d, s = %d, c = %d",
             $time,name,a,b,mode,s,c);
  endfunction
endclass
```

1

rand bit — drive용 입력

`rand`가 붙은 변수만 `randomize()` 때 랜덤 값이 생성된다
TB가 만드는 값이라 x/z가 필요 없어 2상태 `bit`

2

logic — monitor용 출력

DUT 출력은 x · z가 나올 수 있으므로 4상태 `logic`
new 직후 s, c = x (로그의 `s = x`가 이것)

3

constraint — 랜덤 범위 제어

부등호(범위) · `inside`(리스트) · `dist`(확률) 세 가지
모든 constraint는 **동시에 AND**로 적용 → 자세한 건 다음 장

4

debug_print — 상태 출력

`[GEN]` `[DRV]` `[MON]`처럼 어느 단계의 값인지 이름표를 붙여 출력
함수 코드는 Text 영역에 class당 1벌 — 객체가 100개여도 공유

constraint는 AND로 겹친다 — 그래서 128은 한 번도 안 나온다

tb_adder_fork_join.sv 256회 실행 로그에서 a 값과 mode 값을 세어 본 결과

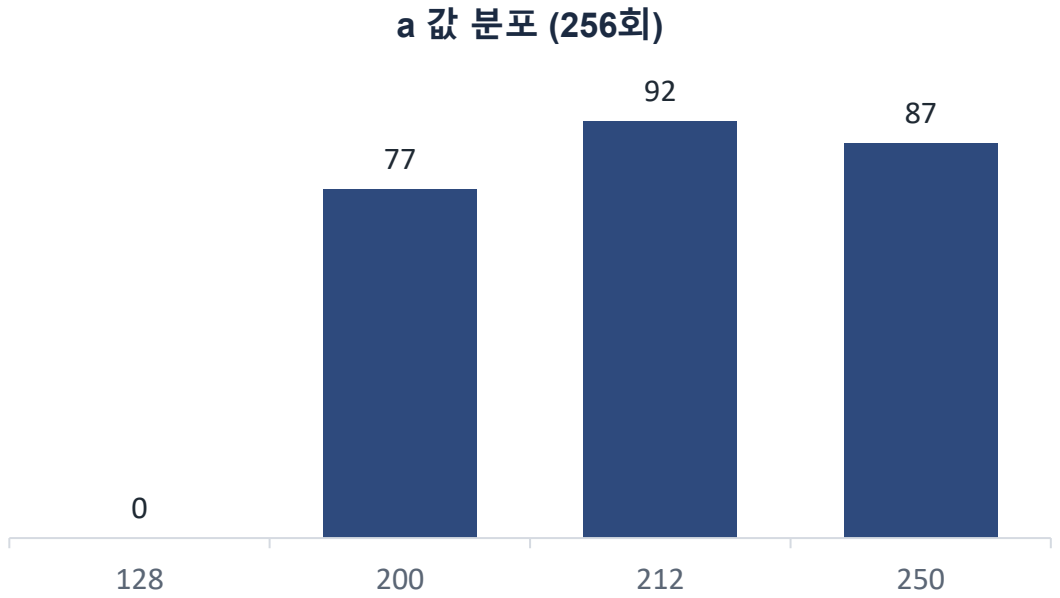
a에 걸린 두 제약의 교집합

`a > 128`	129 ~ 255
`a inside {128,200,212,250}`	128, 200, 212, 250
두 조건 AND	**200, 212, 250 (128 탈락)**

dist 연산자 두 가지

연산자	의미	예: x dist {[1:3]:/60, 4:=40}
`:=`	범위 안 **각 값마다** 가중치	4 → 가중치 40
`:/`	범위 **전체에** 가중치를 나눠 줌	1,2,3 → 각각 20 (60÷3)

이 코드는 `mode dist {0:/80, 1:/20}`처럼 값이 하나씩이라 `:=`와 `:/`의 결과가 같다 (80% : 20%)



80.5%

mode = 0 (206회)

19.5%

mode = 1 (50회)

generator — 랜덤 스티뮬러스 생성

transaction을 만들고 randomize한 뒤 mailbox에 넣는다. scoreboard 판정이 끝나면 다음 것을 만든다

tb_adder_fork_join.sv

```
class generator;

  transaction tr;                                          // 1
  mailbox #(transaction) gen2drv_mbox;
  event scr2gen_evt;

  function new(mailbox #(transaction) gen2drv_mbox_a,      // 2
               event scr2gen_evt);
    this.gen2drv_mbox = gen2drv_mbox_a;
    this.scr2gen_evt = scr2gen_evt;
  endfunction

  task run(int run_count);
    repeat(run_count) begin
      tr = new;                                            // 3
      tr.randomize();
      tr.debug_print("GEN");
      gen2drv_mbox.put(tr);                                // 4
      @(scr2gen_evt);
    end
    #10;
    $display("end gen");
  endtask

endclass
```

1 멤버 = handle 3개

선언만으로는 객체가 없다 (`tr`은 null). 주소를 담을 그릇만 생긴 것

2 new(인자) — 외부 객체 주소를 복사

environment가 만든 mailbox · event의 주소를 내부 handle에 저장

`this.` = 현재 객체의 멤버 → 인자와 이름이 같아도 구분 가능

3 repeat 안에서 매번 new

반복마다 **새 객체**(새 주소)가 생긴다 → 다음 장 메모리 그림

`randomize()`가 rand 변수 a · b · mode에 제약에 맞는 값을 채운다

4 put 후 이벤트 대기

`put`은 시간을 쓰지 않는다 (0ns)

`@(scr2gen_evt)` : scoreboard가 판정을 끝낼 때까지 멈춤 → 한 번에 하나씩 검증

mailbox에 들어가는 건 객체가 아니라 '주소'

tr = new 위치에 따라 heap에서 무슨 일이 일어나는지 (주소는 설명용 예시, 값은 실제 로그)

gen.tr (handle)

주소	값
&gen.tr	0x5000 → 0x6000

gen2drv_mbox 큐 (0x9000)

주소	값
slot 0	0x5000

repeat 1 : tr = new → 0x5000

주소	값
0x5000	헤더
0x5008	a = 250
0x5009	b = 46
0x500A	mode = 0

repeat 2 : tr = new → 0x6000

주소	값
0x6000	헤더
0x6008	a = 212
0x6009	b = 5
0x600A	mode = 0

put(tr) 은 tr 객체 자체를 복사하지 않는다. tr이 들고 있는 **주소값 0x5000**을 큐에 넣는다

다음 반복에서 `tr = new` → tr handle이 0x6000으로 바뀌어도 0x5000 객체는 큐(또는 driver)가 참조하므로 살아 있다

함정 : tr = new 를 repeat 밖에 쓰면?

객체는 0x5000 **하나뿐** → randomize가 같은 객체를 계속 덮어쓴다

mailbox에는 0x5000이 N번 들어간다 → driver가 꺼낼 때마다 **같은 객체(마지막 값)**를 보게 된다

→ 객체를 '보낼 때마다' 새로 만드는 이유

버전별 mailbox 상태 비교

tb_adder.sv : 이벤트 대기 없음 → 0ns에 tr 100개가 한꺼번에 큐에 쌓임 (heap에 100개 동시 존재)

tb_adder_fork_join.sv : put 후 `@(scr2gen_evt)` → 큐에 항상 최대 1개

꺼내서 참조가 모두 끊긴 객체는 GC가 회수

driver — transaction 값을 DUT 핀으로

mailbox에서 꺼낸 값을 virtual interface에 쓰고, 5ns 뒤 monitor에게 '읽어도 된다'고 알린다

tb_adder_fork_join.sv

```
class driver;
  transaction tr;
  mailbox #(transaction) gen2drv_mbox;
  event drv2mon_evt;

1 virtual adder_interface adder_if_v;

  function new(mailbox #(transaction) gen2drv_mbox,
               virtual adder_interface adder_if_v,
               event drv2mon_evt);
    this.adder_if_v = adder_if_v;
    this.gen2drv_mbox = gen2drv_mbox;
    this.drv2mon_evt = drv2mon_evt;
  endfunction

  task run();
    forever begin
2     gen2drv_mbox.get(tr);
      tr.debug_print("DRV");
3     adder_if_v.a = tr.a;
      adder_if_v.b = tr.b;
      adder_if_v.mode = tr.mode;
4     #5;
      ->drv2mon_evt;
      #5;
    end
  endtask
endclass
```

1 virtual interface handle

interface는 정적 HW, class는 동적 → class 안에 HW를 '만들' 수 없다

이미 존재하는 `adder_if`를 **가리키는 참조**만 가진다

2 get(tr) — 꺼내기 (블로킹)

큐 맨 앞 주소를 `drv.tr`에 복사하고 그 slot은 큐에서 빠진다

큐가 비어 있으면 들어올 때까지 대기

3 vif에 쓰기 = DUT 입력

heap의 tr 값이 실체 `adder_if.a/b/mode`로 복사 → DUT가 바로 계산 (조합논리)

4 #5 → 이벤트 → #5

출력이 안정될 시간을 준 뒤 `->drv2mon_evt`로 monitor 깨우기

나머지 #5로 한 transaction = **10ns**

실체 interface와 mailbox는 하나 — 모두 같은 주소를 본다

new의 인자로 '주소'가 전달되므로 gen · drv · mon이 같은 객체 / 같은 HW를 공유한다

adder_if가 driver까지 전달되는 경로

tb_adder
`adder_interface adder_if();`

→

initial
`env = new(adder_if);`

→

environment::new
`drv = new(..., adder_if_v, ...);`

→

driver::new
`this.adder_if_v = adder_if_v;`

vif handle의 상태 변화

시점	adder_if_v 상태
`virtual adder_interface adder_if_v;`	null
`this.adder_if_v = adder_if_v;` (new에서)	실체 adder_if 참조
연결 없이 `adder_if_v.a = 1;`	실행 중 null 접근 오류

adder_if는 정적 영역의 HW **하나**. driver와 monitor의 vif가 모두 같은 실체를 가리키므로, driver가 쓴 값을 monitor가 그대로 읽을 수 있다

handle들이 담고 있는 값 (예시)

주소	값
`env.gen2drv_mbox`	`0x9000`
`gen.gen2drv_mbox`	`0x9000`
`drv.gen2drv_mbox`	`0x9000`

→ **mailbox 객체 @ 0x9000**

순서가 중요 : env.new에서 `gen2drv_mbox = new;`를 gen · drv 생성보다 먼저 해야 한다. 순서가 바뀌면 **null**이 복사되어 put/get에서 오류

monitor — DUT 핀을 읽어 transaction으로 포장

driver의 신호(event)를 받은 순간의 입력과 출력을 새 transaction에 담아 scoreboard로 보낸다

tb_adder_fork_join.sv

```
class monitor;
  transaction tr;
  virtual adder_interface adder_if_v;
  mailbox #(transaction) mon2scb_mbox;
  event drv2mon_evt;

  function new( mailbox#(transaction) mon2scb_mbox,
                virtual adder_interface adder_if_v,
                event drv2mon_evt);
1   this.mon2scb_mbox = mon2scb_mbox; // this : 현재 객체 내부의 핸들
    this.adder_if_v = adder_if_v;     // this + . -> '.' 연결자
    this.drv2mon_evt = drv2mon_evt;
  endfunction

  task run();
    forever begin
2     @(drv2mon_evt);
3     tr = new;
      tr.a = adder_if_v.a;
      tr.b = adder_if_v.b;
      tr.s = adder_if_v.s;
      tr.mode = adder_if_v.mode;
      tr.c = adder_if_v.c;
      tr.debug_print("MON");
4     mon2scb_mbox.put(tr); // monitor에서 생성해서 mailbox로
    end
  endtask
endclass
```

1 this — 인자와 멤버 구분

`this.mon2scb_mbox` = 내 멤버, `mon2scb_mbox` = 인자
이름을 같게 지어도 돼서 `_a` 같은 접미사가 필요 없다

2 @(drv2mon_evt) — 언제 볼지

주기적으로 감시하지 않고 **driver가 값을 바꾼 뒤**에만 샘플링
driver가 #5 기다린 뒤 트리거 → 출력이 안정된 시점

3 새 tr에 입력 + 출력을 함께 담기

driver의 tr를 재사용하지 않고 **새로 new** → 실제 핀에서 읽은 값만 담는다
s · c는 logic이라 DUT가 x를 내면 x 그대로 보인다

4 put → scoreboard

mon2scb_mbox에 주소를 넣으면 scoreboard의 `get`이 깨어난다

scoreboard — 기대값과 비교해 Pass / Fail

DUT와 같은 연산을 TB 쪽에서 따로 계산(reference model)한 뒤 결과를 비교한다

tb_adder_fork_join.sv

```
class scoreboard;
  transaction tr;
  mailbox #(transaction) mon2scb_mbox;
  event scr2gen_evt;
  function new(mailbox #(transaction) mon2scb_mbox,event scr2gen_evt);
    this.mon2scb_mbox = mon2scb_mbox;
    this.scr2gen_evt = scr2gen_evt;
  endfunction
  bit [7:0] expected_s;
  bit expected_c;
  int pass_cnt,fail_cnt;

  task run();
    forever begin
      mon2scb_mbox.get(tr);
      if(tr.mode)
        {expected_c, expected_s} = tr.a - tr.b;
      else
        {expected_c, expected_s} = tr.a + tr.b;
      if({expected_c, expected_s} == {tr.c,tr.s})begin
        $display("%t : pass ",$time);
        pass_cnt ++;
      end else begin
        $display("fail");
        tr.debug_print("SCB");
        fail_cnt ++;
      end
      ->scr2gen_evt;
    end
  endtask
endclass
```

1 **기대값 · 카운터 멤버**

`int` 카운터는 0으로 시작, 끝난 뒤 env가 `scb.pass_cnt`로 읽어 출력

2 **reference model**

DUT의 `assign {c,s}`와 같은 식을 SW로 계산
좌변을 9비트로 묶어야 carry/borrow까지 같이 비교된다

3 **비교 · 판정**

`{c,s}` 9비트 전체가 같으면 pass, 다르면 fail + 그때의 값 출력
참고 : `==`는 x가 섞이면 결과가 x → else(fail)로 감. x까지 정확히 보려면 `===`

4 **->scr2gen_evt — 한 바퀴 완료**

판정이 끝났음을 generator에 알려 다음 transaction 생성을 허락

environment — 부품을 만들고 연결하고 동시에 돌린다

new에서 mailbox · event를 만들어 각 컴포넌트에 나눠 주고, run에서 fork로 4개 task를 병렬 실행

tb_adder_fork_join.sv

```
class environment;
  generator gen;
  driver drv;
  monitor mon;
  scoreboard scb;
  event drv2mon_evt;                                          // 1
  event scr2gen_evt;                                          // 1
  mailbox #(transaction) gen2drv_mbox;                        // 1
  mailbox #(transaction) mon2scb_mbox;                        // 1
  function new(virtual adder_interface adder_if_v);
    gen2drv_mbox = new;                                       // 2
    mon2scb_mbox = new;                                       // 2
    gen = new(gen2drv_mbox,scr2gen_evt);                      // 3
    drv = new(gen2drv_mbox, adder_if_v,drv2mon_evt);          // 3
    mon = new(mon2scb_mbox,adder_if_v,drv2mon_evt);           // 3
    scb = new(mon2scb_mbox,scr2gen_evt);                      // 3
  endfunction

  task run(int run_count);  // argument 선언
    fork                                                      // 4
      gen.run(run_count);                                     // 4
      drv.run();                                              // 4
      mon.run();                                              // 4
      scb.run();                                              // 4
    join_any                                                  // 4
    $display("end fork join");
    #100;
    $display("pass count = %d",scb.pass_cnt);
    $display("fail count = %d",scb.fail_cnt);
  endtask
endclass
```

1 공유 자원 선언

mailbox 2개 · event 2개를 env가 소유. 같은 handle을 넘겨받은 쪽끼리 연결된다

2 mailbox를 먼저 new

주소가 생긴 뒤에 넘겨야 gen · drv가 null이 아닌 실제 객체를 받는다

3 같은 handle = 같은 통로

gen ↔ drv : `gen2drv_mbox` drv ↔ mon : `drv2mon_evt`

mon ↔ scb : `mon2scb_mbox` scb ↔ gen : `scr2gen_evt`

4 fork … join_any

4개 task 동시 시작. drv · mon · scb는 forever라 안 끝나고, **gen이 끝나는 순간** 다음 줄로

#100 여유 후 결과 출력 (파일 이름은 fork_join이지만 실제는 join_any)

tb_adder (top) — HW와 SW가 만나는 곳

정적인 interface · DUT를 깔고, initial에서 동적인 environment를 생성해 실행

tb_adder_fork_join.sv

```
module tb_adder();

1  adder_interface adder_if();
   environment env;

1  adder dut(
     .a(adder_if.a),
     .b(adder_if.b),
     .s(adder_if.s),
     .c(adder_if.c),
     .mode(adder_if.mode)
   );

   initial begin
2    $timeformat(-9,0,"ns",0);
3    env = new(adder_if);
4    env.run(256);

5    $finish;
   end

   endmodule
```

실행 순서

① elaboration (0ns 이전)
adder_if · dut가 정적 영역에 고정. env handle = null

② $timeformat
%t를 ns 단위로 출력 (-9 = 10^{-9}초)

③ env = new(adder_if)
heap에 env 생성 → 안에서 mbox → gen · drv · mon · scb 순서로 new

④ env.run(256)
fork로 4개 task 실행. gen이 256번 끝나면 join_any 통과 → #100 → 결과 출력

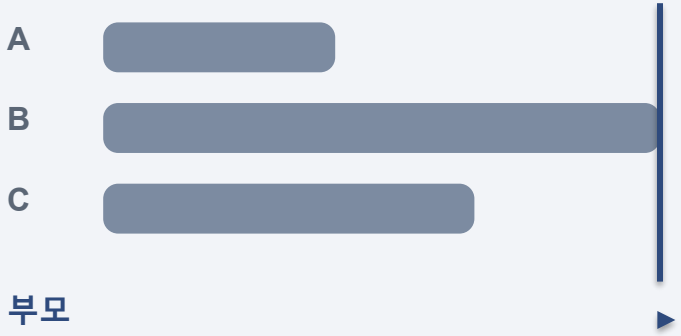
⑤ $finish
시뮬레이션 강제 종료. 이게 없으면 forever task가 남아 종료가 애매해진다

fork 세 가지 — 부모가 언제 다음 줄로 가는가

fork 안의 프로세스들은 동시에 시작한다. 차이는 '부모가 기다리는 조건' 하나 (면접 단골)

`fork … join`

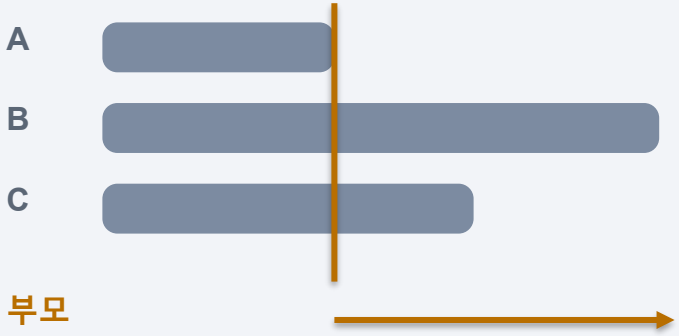
모두 끝나야 진행



진행 시점 : 가장 긴 B가 끝난 뒤

`fork … join_any`

하나라도 끝나면 진행



진행 시점 : 가장 짧은 A가 끝난 뒤

`fork … join_none`

기다리지 않고 바로 진행



진행 시점 : fork 직후 (0 시점)

동시에 시작해도 실행 순서는 보장되지 않는다 — 같은 시각에 깨어난 프로세스들 사이의 순서는 시뮬레이터 마음 (race condition). 순서는 같은 begin…end 안에서만 확정된다

그래서 이 TB는 **event**로 순서를 강제한다. `begin…end` = 순차(blocking), `fork…join` 계열 = 병렬

TB 4개 버전 비교 — env.run()만 바뀌었다

같은 transaction · DUT를 두고 실행 방식만 바꿔 가며 발전시킨 과정

파일	env.run() 핵심 구조	동작 방식	실제 결과 (xsim)

tb_adder.sv	`forever begin`
	`  gen.run();`
	`  drv.run();`
	`end`
		gen이 0ns에 100개 put → drv가 10ns마다 하나씩. mon · scb 없음	DRV 0~990ns. drv.run()이 끝나지 않아 **$finish 미도달** (이벤트 소진으로 정지)

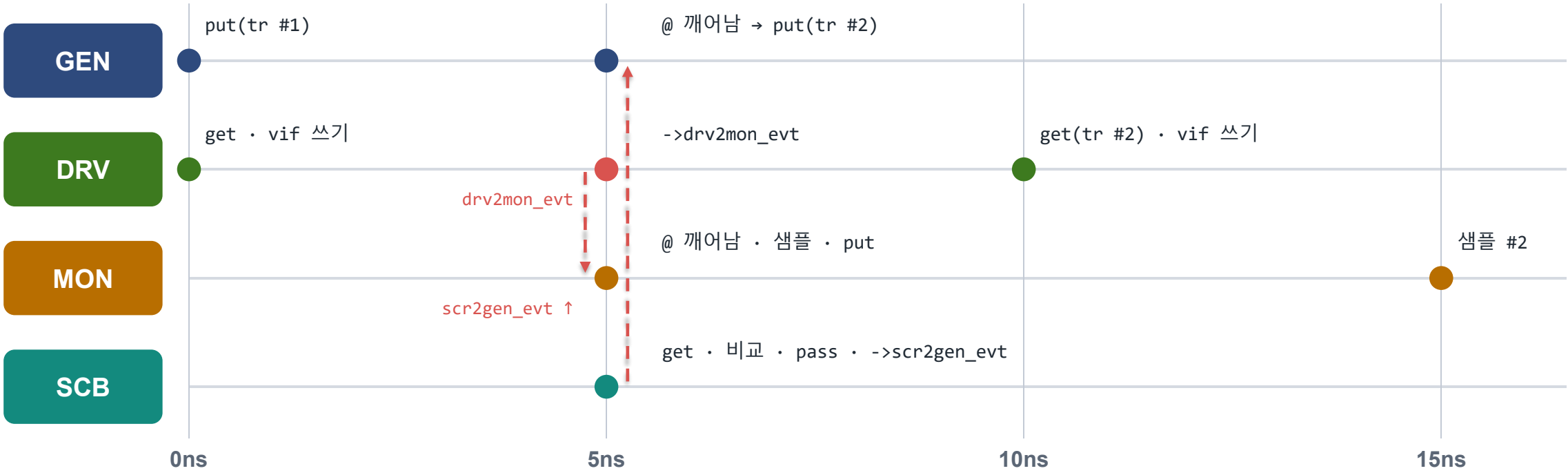
tb_adder_sequential.sv	`repeat(run_count) begin`
	`  gen.run(); drv.run();`
	`  mon.run(); scb.run();`
	`end`
		각 run이 1개씩 처리하고 끝남. 한 번에 한 컴포넌트만 동작 (순차)	10회 → **pass 10 / fail 0**, 100ns 종료. MON은 drv #10 뒤에 샘플

tb_adder_fork_join_none.sv	`fork`
	`  gen.run(run_count);`
	`  drv.run();`
	`join_none`
		부모가 기다리지 않음. mon · scb 주석 처리	**"0ns : end fork join"이 GEN보다 먼저** 출력. pass 0 / fail 0, 100ns 종료

tb_adder_fork_join.sv	`fork`
	`  gen.run(run_count); drv.run();`
	`  mon.run(); scb.run();`
	`join_any`
		4개 병렬 + event 2개로 핸드셰이크	256회 → **pass 256 / fail 0**, $finish @ 2665ns

event 핸드셰이크 — 한 transaction이 도는 10ns

tb_adder_fork_join.sv 로그의 시각 그대로. 다음 GEN은 scoreboard 판정 직후, 다음 DRV는 #5 뒤에 시작



왜 5ns에 샘플?
drv가 값을 쓰고 #5 기다린 뒤 트리거 → 조합논리 출력이 확정된 뒤 읽는다

왜 다음 DRV는 10ns?
GEN은 5ns에 put 하지만 drv는 남은 #5를 마저 기다린 뒤 get

결과
transaction 1개 = 10ns → 256개 = 2560ns. 로그 첫 줄들: 0 / 5 / 15 / 25ns

결과 ① tb_adder.sv · tb_adder_sequential.sv

xsim -R 실행 로그 발췌 (Vivado 2020.2)

tb_adder.sv (GEN 100 / DRV 100)

```
0ns :[GEN] a =189, b = 221, mode = 1, s =   x, c = x
0ns :[GEN] a =173, b = 134, mode = 0, s =   x, c = x
0ns :[GEN] a =170, b =  42, mode = 1, s =   x, c = x
...  (0ns에 GEN 100줄)
0ns :[DRV] a =189, b = 221, mode = 1, s =   x, c = x
10ns :[DRV] a =173, b = 134, mode = 0, s =   x, c = x
...
980ns :[DRV] a = 58, b =  45, mode = 0, s =   x, c = x
990ns :[DRV] a =119, b = 176, mode = 1, s =   x, c = x
exit          ← $finish 메시지 없음
```

tb_adder_sequential.sv (run 10)

```
0ns :[GEN] a =200, b =   6, mode = 0, s =   x, c = x
0ns :[DRV] a =200, b =   6, mode = 0, s =   x, c = x
10ns :[MON] a =200, b =   6, mode = 0, s = 206, c = 0
10ns : pass
10ns :[GEN] a =200, b =   5, mode = 1, s =   x, c = x
10ns :[DRV] a =200, b =   5, mode = 1, s =   x, c = x
20ns :[MON] a =200, b =   5, mode = 1, s = 195, c = 0
20ns : pass
...
100ns : pass
pass count =          10
fail count =           0
$finish called at time : 100 ns
```

관찰

- GEN이 전부 0ns — put은 시간을 안 쓰고, 대기도 없어서 100개가 한꺼번에 생성
- drv.run()이 forever라 반환하지 않음 → env.run 다음 줄 `$finish`에 못 감

관찰

- GEN · DRV는 같은 시각, MON은 drv의 #10이 끝난 뒤 → 한 바퀴 10ns
- 순서는 확실하지만 한 번에 한 컴포넌트만 일한다 (병렬성 없음)

결과 ② fork_join_none · fork_join(join_any)

같은 seed라 두 로그의 첫 transaction(a=250, b=46)이 같다

tb_adder_fork_join_none.sv (run 10)

```
0ns : end fork join
0ns :[GEN] a =250, b =  46, mode = 0, s =   x, c = x
0ns :[GEN] a =200, b =  29, mode = 1, s =   x, c = x
...  (0ns에 GEN 10줄)
0ns :[DRV] a =250, b =  46, mode = 0, s =   x, c = x
10ns : end gen
10ns :[DRV] a =200, b =  29, mode = 1, s =   x, c = x
...
90ns :[DRV] a =212, b =  12, mode = 0, s =   x, c = x
pass count =           0
fail count =           0
$finish called at time : 100 ns
```

tb_adder_fork_join.sv (run 256)

```
0ns :[GEN] a =250, b =  46, mode = 0, s =   x, c = x
0ns :[DRV] a =250, b =  46, mode = 0, s =   x, c = x
5ns :[MON] a =250, b =  46, mode = 0, s =  40, c = 1
5ns : pass
5ns :[GEN] a =212, b =   5, mode = 0, s =   x, c = x
10ns :[DRV] a =212, b =   5, mode = 0, s =   x, c = x
15ns :[MON] a =212, b =   5, mode = 0, s = 217, c = 0
...
2555ns : pass
end gen
end fork join
pass count =         256
fail count =           0
$finish called at time : 2665 ns
```

관찰

- join_none → 자식이 돌기 전에 부모가 먼저 `end fork join`을 찍는다. 자식은 부모가 `#100`으로 멈출 때 실행
- mon · scb를 안 돌려 pass 0 — 검증이 아니라 '구동만' 한 상태

관찰

- 250+46=296 → s=40, c=1 : 9비트 carry까지 정확히 일치
- 2555 판정 → gen `#10` → 2565 join_any 통과 → `#100` → **2665ns** $finish

최종 버전 결과 요약 — 256회 전부 통과

tb_adder_fork_join.sv · run_count = 256

256	0	10ns	2665ns
pass	fail	transaction 1개당	$finish 시각

숫자 해석

항목	값	이유
carry 발생 (c=1)	**96회**	a ≥ 200, b < 64 → 덧셈 합이 256을 넘는 경우가 많다. 뺄셈은 a > b라 borrow 0
a = 128	**0회**	`a > 128` 과 `inside {128,...}`가 AND → 128은 해집합에 없음
mode = 1	**50회 (19.5%)**	`dist 1:/20`이 목표 20% → 랜덤이라 근사값
총 시간	**2665ns**	256 × 10ns = 2560 → 마지막 판정 2555ns + gen #10 + env #100

주의할 점 — 이번 코드에서 찾은 버그 포인트

지금은 결과가 맞게 나오지만, 코드를 바꾸면 바로 터질 수 있는 곳

! $finish 미도달

tb_adder.sv

`forever begin gen.run(); drv.run(); end` — drv.run()이 끝나지 않아 다음 줄로 못 감. 시뮬 종료는 항상 `$finish`로 명시

! new 위치

generator

`tr = new`가 repeat 밖이면 같은 객체를 덮어씀 → mailbox에 같은 주소 N개

! new 순서

environment

mailbox보다 gen · drv를 먼저 new하면 null handle이 복사됨 → put/get 실행 중 오류

! randomize 실패 무시

generator

제약이 모순이면 `randomize()`가 0을 반환하는데 확인 안 함 → `if(!tr.randomize()) $fatal(...)`

! event 놓침

scb → gen

`->evt` 순간에 `@(evt)`로 기다리는 쪽이 없으면 신호가 사라짐. 같은 시각 경쟁이면 `wait(evt.triggered)`

! 반환형 없는 함수

transaction

`function debug_print` = 암묵적 1비트 반환 함수. 값을 안 돌려주면 `function void`가 정석

면접 단골 질문 — 한 줄로 답해 보기

답을 가리고 먼저 말해 본 뒤 확인

Q1 fork-join / join_any / join_none 차이는?

부모가 **모두 / 하나 / 아무것도** 안 기다리고 다음 줄로 간다. 자식은 모두 동시에 시작

Q2 virtual interface를 쓰는 이유는?

class는 동적이라 정적 HW(interface)를 안에 만들 수 없음 → 이미 있는 실체를 **참조**

Q3 mailbox와 event의 차이는?

mailbox는 **데이터**(handle)를 전달하는 큐, event는 데이터 없이 **타이밍**만 알리는 신호

Q4 handle과 object의 차이는?

handle은 주소를 담는 변수(초기 null), object는 `new`로 heap에 생긴 실체

Q5 transaction에서 bit와 logic을 나눈 이유는?

입력은 TB가 만드는 값이라 2상태 `bit`, 출력은 DUT가 x·z를 낼 수 있어 4상태 `logic`

Q6 같은 시각 프로세스의 실행 순서는?

보장 안 됨 (race). 순서가 필요하면 event · 지연 · 같은 begin…end 로 강제

PART 5 · 복습

한 장 요약

transaction	데이터 묶음. rand bit = 입력, logic = 출력, constraint로 범위 제어
generator	new → randomize → put → 판정 이벤트 대기
driver	get → virtual interface에 쓰기 → #5 → ->drv2mon_evt
monitor	@(drv2mon_evt) → 핀 읽어 새 tr → put
scoreboard	get → 기대값 계산 → {c,s} 비교 → ->scr2gen_evt
environment	mailbox · event 생성 후 나눠 주기 → fork … join_any
tb_adder	interface · DUT(정적) + initial에서 env new · run · $finish

데이터는 mailbox · 타이밍은 event · 하드웨어는 virtual interface